R Programming Basics

Big Data Analytics Using R · Unit 4 · Exam Prep Notes (easy language)
Based on Unit_4 slides. Parts marked "extra" are short additions where the slides had only a heading or a table.

> **Unit 4 at a glance:** R & RStudio → Variables → print/cat → Data types → Vectors → Lists → Matrices → Arrays → Data frames → ANOVA → Covariance & Correlation.

1. RStudio and Installing R

- **R** = programming language for statistics, data analysis and graphs.
- **RStudio (now Posit)** = an **IDE** (Integrated Development Environment) for R. It gives an easy screen to write and run code, make plots, reports and manage packages. Works with R Markdown, Shiny apps and Python.

4 panes of RStudio

Pane	Use
Source pane	Write R scripts
Console	Commands run here
Environment / History	Shows variables, datasets and past commands
Files / Plots / Packages / Help	Shows files, graphs, installed packages, help

Installation steps

1. **Install R** from `cran.r-project.org` → choose OS (Windows / macOS / Linux) → run installer → Next until finish.
2. **Install RStudio Desktop (free)** from `posit.co/download/rstudio-desktop` → download for your OS → run installer.
3. **Check:** open RStudio, type `version` in the console. If R version details appear, setup is successful.

> **Remember:** install R first, then RStudio (RStudio needs R).

2. Variables, print() and cat()

How to create a variable (3 ways)

- `<-` (most commonly used) e.g. `x <- 5`
- `=` (also works)
- `->` (less common, assigns right to left) e.g. `5 -> x`

Rules for naming variables

- **Allowed:** letters, numbers, dot (.) and underscore (_).
- **Not allowed:** starting with a number; spaces.
- R is **case-sensitive**: `Age` and `age` are different.

print() vs cat()

print()	cat()
Displays R objects (numbers, strings, vectors, lists). Mostly used for debugging / to see internal structure. **Adds quotes** around strings and shows index `[1]`.	Stands for **concatenate and print**. Gives formatted, clean output (messages, reports). **No** `[1]` or quotes. Use `\n` for a new line.

```
# Even or odd program
num <- as.numeric(readline(prompt = "Enter a number: "))
if (num %% 2 == 0) {
  cat(num, "is an even number.\n")
} else {
  cat(num, "is an odd number.\n")
}
```

3. Data Types in R

R is a **vector-based** language. It has **6 basic (atomic) data types:** Numeric, Integer, Complex, Logical, Character, Raw.

Type	Example	Key points
Numeric	`x <- 25.5`	**Default** type for numbers; decimal values. Even `20` is numeric (not integer) unless specified.
Integer	`y <- 10L`	Whole numbers; must use suffix **L**. Convert with `as.integer(5.9)` → 5.
Character	`name <- "reva"`	Text/strings inside `" "` or `' '`.
Logical	`flag <- TRUE`	Boolean: `TRUE` / `FALSE`. Used in conditions, comparisons, filtering.
Complex	`z <- 2 + 3i`	Complex numbers `a + bi`; used in scientific work.
Raw	`charToRaw("ABC")`	Stores raw bytes; rarely used; for binary data (files, images).

Use `class(x)` to check the type. e.g. `class(10L)` gives "integer", `class(4+3i)` gives "complex".

4. Vectors

- A **vector** is the simplest data structure: a **one-dimensional** collection of elements.
- All elements must be of the **same type** (homogeneous). Created using `c()`.
- Like arrays in R. **Indexing starts from 1** (not 0 like C/Python).
- Types: numeric, integer, character, logical, complex, raw vectors.

```
num_vec  <- c(10, 20, 30, 40)          # numeric
int_vec  <- c(10L, 20L, 30L)           # integer
char_vec <- c("apple", "banana")       # character
log_vec  <- c(TRUE, FALSE, TRUE, NA)   # logical (NA = missing)
comp_vec <- c(3+2i, 4-5i)              # complex
```

Type coercion: if a vector has mixed types, R converts all to one type. `c('geeks', '2', 'hello', 57)` becomes **character** because characters are more general than numbers. `typeof(c(4,5,6))` gives "double"; with `L` gives "integer".

Methods to create a vector (6 ways)

Method	Example and result
1. `c()` combine	`v1 <- c(10,20,30,40)`
2. `:` colon	`1:10` → 1 2 3 ... 10
3. `seq()`	`seq(1,10)`; `seq(1,20,by=2)` → 1 3 5 ... 19; `seq(1,10,length.out=5)` → 1 3.25 5.5 7.75 10
4. `rep()`	`rep(5,times=4)` → 5 5 5 5 `rep(c(1,2,3),times=3)` → 1 2 3 1 2 3 1 2 3 `rep(c(1,2,3),each=2)` → 1 1 2 2 3 3
5. `vector()`	`vector("numeric",length=5)` → 0 0 0 0 0; `vector("logical",4)` → FALSE x4
6. Constructors	`numeric(4)` → 0 0 0 0; `character(3)` → "" "" ""; `logical(5)` → FALSE x5

Vector operations

Operation	Code / explanation
Length	`length(x)` gives the number of elements (`c(1,2,3,4,5)` → 5).
Access	`X[2]` gives 2nd element. `Y[c(4,1)]` gives 4th and 1st elements.
Modify	`X[3] <- 1` changes 3rd element. `X[1:5] <- 0` changes first five to 0. `X <- X[c(3,2,1)]` keeps/reorders chosen elements.
Delete	`M <- NULL` deletes all elements of the vector.
Sort	`sort(X)` ascending; `sort(X, decreasing = TRUE)` descending. For `c(8,2,7,1,11,2)`: 1 2 2 7 8 11 and 11 8 7 2 2 1.

5. List (extra)

The slides have only a "LIST" heading with no content. Short notes so you can answer a basic question:

- A **list** is a collection that can hold elements of **different types** (numbers, text, vectors, even other lists). Vector = same type; list = mixed types.
- Created with `list()`, e.g. `l <- list(1, "a", TRUE, c(1,2,3))`. Access with `l[[2]]` or by name `l$name`.

6. Matrix

A **matrix** is a **two-dimensional** structure with elements of the **same data type** arranged in rows and columns.

Creating a matrix

```
matrix(data, nrow, ncol, byrow = FALSE, dimnames)
m <- matrix(c(1,2,3,4,5,6), nrow = 2, ncol = 3)
#      [,1] [,2] [,3]
# [1,]    1    3    5
# [2,]    2    4    6
```

- **Default is column-wise filling.** Use `byrow = TRUE` to fill row-wise (gives 1 2 3 / 4 5 6).
- Character matrix: `matrix(c("A","B","C","D"), nrow=2)`. Logical matrix: `matrix(c(TRUE,FALSE,TRUE,FALSE), nrow=2)`.
- Naming: `rownames(m) <- c("Row1","Row2")`; `colnames(m) <- c("Col1","Col2","Col3")`. Or use `dimnames`.
- Vector to matrix: `matrix(c(1,2,3,4,5,6), nrow = 3)`.

Special matrices

Type	Code	Result
Constant matrix	`matrix(5, 3, 3)`	3x3 matrix with all elements 5
Diagonal matrix	`diag(c(5,3,3), 3, 3)`	5, 3, 3 on the diagonal, 0 elsewhere
Identity matrix	`diag(1, 3, 3)`	1s on diagonal, 0 elsewhere

Matrix information (metrics)

```
A <- matrix(c(1,2,3,4,5,6,7,8,9), nrow=3, ncol=3, byrow=TRUE)
dim(A)      # 3 3  (dimensions)
nrow(A)     # 3    (number of rows)
ncol(A)     # 3    (number of columns)
length(A)   # 9    (number of elements)  = prod(dim(A))
t(A)        # transpose
```

Accessing elements: `m[row, column]`

Code (using A above)	Result
`A[1, 2]` (row 1, col 2)	2
`A[2, 3]`	6
`A[1:2, ]` (rows 1 and 2, all columns)	first two rows
`A[, 1:2]` (all rows, columns 1-2)	first two columns
`A[1:3, 1:2]` (submatrix)	rows 1-3, columns 1-2

Rule: value **before** the comma = rows, value **after** the comma = columns. Empty means "all".

Modify, add and delete

Task	Code
Modify element	`A[3, 3] <- 30` (3rd row, 3rd column becomes 30)
Add a row	`C <- rbind(A, B)` where `B <- matrix(c(10,11,12), nrow=1)`
Add a column	`C <- cbind(A, B)` where `B` is a 3x1 matrix; or `cbind(number, new_column)`
Delete a row	`A <- A[-2, ]` (deletes 2nd row; **negative sign** = delete)
Delete a column	`A <- A[, -2]` (deletes 2nd column)

> **Dimension rule:** `rbind` needs the same number of **columns**; `cbind` needs the same number of **rows**. If not, error: "*number of rows of matrices must match*" (e.g. cbind of a 3x3 and a 1x3 matrix).

Matrix operations (extra): `A + B` (add), `A * B` (element-wise multiply), `A %*% B` (matrix multiply), `A * 3` (each element x 3), `solve(A)` (inverse), `t(A)` (transpose), `diag(A)` (diagonal elements).

Practice questions from the slides, with answer codes

The slides give only the questions. These solutions were written by me and not run in R here, so test them in RStudio.

Q1. A = [1 1 3; 5 2 6; -2 -1 -3]

```
A <- matrix(c(1,1,3, 5,2,6, -2,-1,-3), nrow = 3, byrow = TRUE)
A <- cbind(A, c(10, 6, 2))     # i) add new column (10 6 2)
A[, 3] <- A[, 2] + A[, 3]      # ii) 3rd column = 2nd + 3rd column
A[3, 3] <- 9                   # iii) direct assignment to 3rd row, 3rd column
```

Q2. D = [1 2; 3 4], E = [5 6; 7 8]

```
D <- matrix(c(1,2,3,4), nrow = 2, byrow = TRUE)
E <- matrix(c(5,6,7,8), nrow = 2, byrow = TRUE)
D + E      # a) add
D %*% E    # b) matrix multiplication
D * 3      # c) multiply each element by 3
```

Q3. 4x4 matrix with values 1 to 16

```
M <- matrix(1:16, nrow = 4, ncol = 4)
M[3, 2]        # a) 3rd row, 2nd column
M[, 4]         # b) entire 4th column
M[1:2, 3:4]    # c) rows 1-2, columns 3-4
```

Q4. 5x5 matrix with values 1 to 25

```
M <- matrix(1:25, nrow = 5)
diag(M)                      # a) diagonal elements
M[2, ] <- c(10,20,30,40,50)    # b) replace 2nd row
M[M > 15]                    # c) all elements greater than 15
```

Q5. C = [2 4 6; 1 3 5; 7 8 9]

```
C <- matrix(c(2,4,6, 1,3,5, 7,8,9), nrow = 3, byrow = TRUE)
C1 <- C; C1[1, ] <- c(10, 20, 30)        # a) replace first row
C2 <- C[, -2]                            # b) delete second column
C3 <- rbind(C, c(11, 12, 13))            # c) add new row at the bottom
```

Each part is done on the original C. Remember: a new row must have as many values as the matrix has columns.

Q6. A = [2 4 6; 1 3 5; 7 8 9], B = [4 2; 6 8], C = [1 3; 5 7]

```
A <- matrix(c(2,4,6, 1,3,5, 7,8,9), nrow = 3, byrow = TRUE)
B <- matrix(c(4,2,6,8), nrow = 2, byrow = TRUE)
C <- matrix(c(1,3,5,7), nrow = 2, byrow = TRUE)

A1 <- rbind(A, c(11, 12, 13))     # a) add new row (11 12 13)
A2 <- A[, -1]                     # b) delete the first column
A3 <- A; A3[, 3] <- A[, 1] + A[, 2]  # c) 3rd column = 1st + 2nd column

B * C        # a) element-wise multiplication
B %*% C      # b) matrix product B x C
solve(B)     # c) inverse of B
```

Q7. 4x4 matrix M = [5 9 3 7; 2 8 1 4; 6 0 12 10; 11 14 13 15]

```
M[M %% 2 == 0]        # a) all even numbers
M[c(1, 3), ]          # b) elements in 1st and 3rd rows
M[M < 5] <- 0          # c) replace values less than 5 with 0
```

Q8. 4x4 matrix filled by column with 20 to 35

```
M <- matrix(20:35, nrow = 4, ncol = 4)
M[c(1,4), ] <- M[c(4,1), ]       # a) swap 1st and 4th rows
rowSums(M); colSums(M)           # b) row sums and column sums
M <- cbind(M, M[,1] * M[,2])     # c) new column = product of 1st and 2nd columns
```

7. Array

- An **array** stores data in a **fixed number of dimensions**; all elements are of the **same type**.
- 1-D array = **vector**; 2-D array = **matrix**; 3-D or more = array.
- Created with `array(data, dim = c(nrow, ncol, nmat), dimnames = names)`: `nrow` = rows, `ncol` = columns, `nmat` = number of matrices, `dimnames` = names (default NULL; otherwise a list of names).

```
vec1 <- c(1,2,3,4,5,6,7,8,9)           # 1-D array (vector)
length(vec1)                            # 9

arr <- array(2:13, dim = c(2, 3, 2))    # 2 rows x 3 cols x 2 matrices
dim(arr)                                # 2 3 2

# Using vectors: total elements must equal 2x3x2 = 12
arr <- array(c(vec1, c(10,11,12)), dim = c(2,3,2))

# Naming
row_names <- c("row1","row2"); col_names <- c("col1","col2","col3")
mat_names <- c("Arr1","Arr2")
arr <- array(2:14, dim = c(2,3,2), dimnames = list(row_names, col_names, mat_names))
```

> **Tip:** `dim(arr)` returns `2 3 2`: first dimension (rows) = 2, second (columns) = 3, third (matrices) = 2. If data is shorter than needed, R repeats it; extra data is dropped.

8. Data Frames

- A **data frame** stores **tabular data** (rows and columns), like an Excel sheet or SQL table.
- Each **column can have a different type** (character, numeric, logical, factor). Each column is a vector.
- Columns have names; **all columns must have the same number of rows**.

Matrix	Data frame
All elements same type	Different types in different columns
Made with `matrix()`	Made with `data.frame()`

Common operations

```
student <- data.frame(
  id   = c(1:5),
  name = c("Sachin","Sourav","Dravid","Sehwag","Dhoni"),
  age  = c(53,52,62,60,51)
)
print(student)        # 1. create and print
str(student)          # 2. structure (types of each column)
summary(student)      # 3. statistical summary (min, max, mean, quartiles...)
student$name          # 4. extract a column by name
student$location <- c("Kolkata","Delhi","Bangalore","Hyderabad","Chennai")  # 5. add column
student[1]            # 6. access: first column as data frame
student[["name"]]     #    column using [[ ]]
student$age           #    column using $
dim(student)          # 7. number of rows and columns  -> 5 4
```

Function	Purpose
`str()`	Shows the structure: type and content of each column
`summary()`	Gives statistical summary of each column
`dim()`	Gives rows and columns count
`$`, `[ ]`, `[[ ]]`	Three ways to access columns

Add rows and columns

```
Products <- data.frame(Product_ID = c(101,102,103),
  Product_Name = c("T-Shirt","Jeans","Shoes"),
  Price = c(15.99,29.99,49.99), Stock = c(50,30,25))

# Add a row with rbind()
New_Product <- c(104, "Sunglasses", 39.99, 40)
Products <- rbind(Products, New_Product)

# Add a column with cbind()
Discount <- c(5, 10, 8)
Products <- cbind(Products, Discount)
colnames(Products)[ncol(Products)] <- "Discount"
```

Exam note: `c(104, "Sunglasses", 39.99, 40)` mixes types, so R turns it into a character vector. Adding it with `rbind` may turn number columns into text. (Extra remark from vector coercion rule.)

Remove rows and columns

```
data <- data.frame(friend_id = c(1,2,3,4,5),
  friend_name = c("Sachin","Sourav","Dravid","Sehwag","Dhoni"),
  location = c("Kolkata","Delhi","Bangalore","Hyderabad","Chennai"))

data <- subset(data, friend_id != 3)   # remove row where friend_id = 3

library(dplyr)
data <- select(data, -location)         # remove column (minus sign = remove)
```

Combine data frames

Way	Code and rule
Vertically (stack rows)	`rbind(df1, df2)` — both must have the **same column names** (e.g. Name, Age, Score). Works for two or more data frames.
Horizontally (side by side)	`cbind(df1, df2)` — both must have the **same number of rows**. e.g. (Name, Age, Score) + (Height, Weight).

9. ANOVA (Analysis of Variance) in R

- **ANOVA** checks whether the **means of different groups are significantly different**, by comparing the variation **between** groups with the variation **within** groups.
- Relates a **categorical** variable (groups) to a **continuous** variable. Used in business, biology, social science and experiments.

Hypotheses

- **Null (H_0):** all group means are equal (no effect of the categorical variable).
- **Alternative (H_1):** at least one group mean is different (the categorical variable has an effect).

Types

One-way ANOVA	Two-way ANOVA
One categorical independent variable + one continuous dependent variable	Two categorical independent variables + one continuous dependent variable; can also detect **interaction** between the two factors

Implementation steps (mtcars dataset)

```
install.packages("dplyr"); library(dplyr)      # 1. install and load packages
head(mtcars)                                    # 2. view first rows

# 3. One-way: does mean displacement (disp) differ across gear levels?
mtcars_aov <- aov(mtcars$disp ~ factor(mtcars$gear))
summary(mtcars_aov)

# 4. Two-way: effect of gear and am (transmission) on disp
mtcars_aov2 <- aov(mtcars$disp ~ factor(mtcars$gear) * factor(mtcars$am))
summary(mtcars_aov2)

# 5. Best-fit model using AIC
install.packages("AICcmodavg"); library(AICcmodavg)
model.set   <- list(mtcars_aov, mtcars_aov2)
model.names <- c("mtcars_aov", "mtcars_aov2")
aictab(model.set, modnames = model.names)

# 6. Visualize with ggplot2 boxplots (two plots side by side with gridExtra)
ggplot(mtcars, aes(x = factor(gear), y = disp, fill = factor(gear))) +
  geom_boxplot() + labs(title="One-Way ANOVA", x="Gear", y="Displacement") + theme_minimal()
```

Functions used

Function	Use
`install.packages()`	Install a package from CRAN
`library()`	Load a package into the session
`head()`	Show first few rows
`aov()`	Perform ANOVA
`factor()`	Convert a variable into a categorical factor
`summary()`	Display the ANOVA table
`aictab()` (AICcmodavg)	Compare models by AIC score
`ggplot()`, `geom_boxplot()`, `labs()`, `theme_minimal()`	Create boxplot with title, labels and minimal theme
`grid.arrange()` (gridExtra)	Place multiple plots side by side

Reading the ANOVA output table

Term	Meaning
Df	Degrees of freedom
Sum Sq	Sum of squares within and between groups
Mean Sq	Mean of the sum of squares
F value	Ratio of variance **between** groups to variance **within** groups
Pr(>F)	P-value: statistical significance

Results in the slides: One-way ANOVA: **gear** has a significant effect on disp ($p < 0.05$). Two-way ANOVA: gear is significant, but **am** is not. AIC: the **simpler model (mtcars_aov)** fits better (lower AIC is better). Boxplots show differences in displacement across gear levels.

Extra: if p-value < 0.05, reject H_0 (group means differ). If p-value ≥ 0.05, do not reject H_0.

10. Covariance and Correlation

- Both measure the **linear relationship** between two variables (bivariate data), but capture different things.
- **Covariance** = how two variables **change together**. **Correlation** = **strength and direction** of the linear relationship.

Feature	Covariance	Correlation
Definition	Shows how two variables change together	Shows strength and direction of linear relationship
Scale	Not standardized; depends on units of the variables	Standardized
Value range	Any value, from negative infinity to positive infinity	Always between **-1 and 1**
Comparability	Difficult to compare across datasets	Easy to compare across datasets
Use cases	Risk analysis, portfolio management	Data analysis, regression, prediction models

Extra (R functions): `cov(x, y)` for covariance and `cor(x, y)` for correlation. Correlation near **+1** = strong positive, near **-1** = strong negative, near **0** = no linear relationship.

11. Last-Minute Revision

- **RStudio** = IDE for R. 4 panes: Source, Console, Environment/History, Files/Plots/Packages/Help. Install R first.
- **Assign:** `<-`, `=`, `->`. Names: no space, can't start with a number, case-sensitive.
- **print()** shows quotes and [1]; **cat()** gives clean output.
- **6 data types:** Numeric (default), Integer (L), Complex, Logical, Character, Raw.
- **Vector:** 1-D, same type, `c()`, index starts at 1. Create with c, :, seq, rep, vector, constructors.
- **Matrix:** 2-D, same type, `matrix()`, column-wise by default (`byrow=TRUE` for rows). Delete with negative index. rbind = add row, cbind = add column.
- **Array:** multi-dimensional, `array(data, dim=c(r,c,n))`, same type.
- **Data frame:** table, different types per column, equal rows. `str()`, `summary()`, `dim()`, `$`.
- **ANOVA:** `aov()`; H_0 = all means equal; one-way (1 factor) vs two-way (2 factors); F value and Pr(>F).
- **Covariance** = any value, depends on units; **correlation** = -1 to 1, standardized.